

## **CS202-Lab2 – Report**

**Hunter Adomitis(hadom001)**

**Iftikhar Kalanther (ikala005)**

**Joseph D'Angelo(jdan003)**

We created and tested the system calls (sched\_statistics and sched\_tickets), lottery scheduling, and stride scheduling as instructed in the lab2 document. The entire XV6 source code with our modifications is available in the GitHub repository (<https://github.com/Hadom-515/CS202-lab.git>). The video demo for this lab2 assignment is available at <https://drive.google.com/file/d/1fx7iAb5A5FiKFJDCUO2ril9HnrzD1ZUP/view?usp=sharing>

## 1. Modified files:

The following files were modified to create and make sched\_statistics and sched\_tickets syscalls.

### kernel folder:

1. defs.h
2. proc.c
3. proc.h
4. syscall.c
5. syscall.h
6. sysproc.c

### user folder:

1. lab2.c
2. user.h
3. usys.pl

### CS202-lab:

1. Makefile

## 2. Changes and screenshots:

- kernel/syscall.h:

syscall.h was modified to define new numbers for the syscalls – sched\_statistics(25) and sched\_tickets(26) - as shown in the screenshot below.

```
26      #define SYS_sched_statistics 25
27      #define SYS_sched_tickets 26
```

- kernel/syscall.c:

- Declared sched\_statistics and sched\_tickets.

```

110      extern uint64 sys_sched_statistics(void);
111      extern uint64 sys_sched_tickets(void);

```

- Updated the system call table (pointer array) \*syscalls[] with sched\_statistics and sched\_tickets system calls.

```

139      [SYS_sched_statistics] sys_sched_statistics,
140      [SYS_sched_tickets] sys_sched_tickets
-

```

- **kernel/sysproc.c (sys\_sched\_statistics):**

- Defined syscall function 'sys\_sched\_statistics'. This function calls print\_statistics() defined in kernel/proc.c, which will print the PID, Name, ticket value and total number of times called for scheduling for each process.

```

uint64 sys_sched_statistics(void){
    print_statistics();
    return 0;
}

```

- **kernel/sysproc.c (sys\_sched\_tickets):**

- Defined syscall function 'sys\_sched\_tickets'.
- The current process information is retrieved by defining a pointer to the proc structure and calling the function myproc(), which returns the current process's proc structure pointer.
- This syscall function validates the number of tickets passed by the user. If it is less than or equal to 10,000, the same will be allocated to the respective process. If more than 10,000, then the process will be allocated only 10,000, as the number of tickets allocated cannot exceed 10,000.
- If the number of tickets to be allocated is less than zero, this syscall will return -1.
- The stride value is calculated by dividing 10,000 (given K value) by the number of tickets allocated to each process and the pass value is initialized with this stride value.

```

158     uint64 sys_sched_tickets(void){
159         int n;
160         argint(0,&n);
161         struct proc* p = myproc();
162         if(n<0){
163             return -1;
164         }
165         if(n<=10000){
166             p->tickets=n;
167         }
168         else{
169             p->tickets=10000;
170         }
171         p->stride=10000/p->tickets;
172         p->pass=p->stride;
173         return 0;
174     }

```

- kernel/proc.h:

- The proc struct definition in proc.h is updated to include 4 variables – tickets, stride, pass and ticks. ‘tickets’ will store the number of tickets allocated to this process. ‘ticks’ will hold the number of times the process is scheduled to run. The stride and pass values are stored in stride and pass, respectively.

```

94         int tickets;
95         int stride;
96         int pass;
97         int ticks;

```

---

- kernel/proc.c:

- The allocproc() function is modified to initialize the above 4 variables. The 'tickets' is initialized to 10,000, and stride to 10,000(K) by tickets, and pass is initialized to stride value. 'ticks' is initialized to 0.

```
128         p->tickets=10000;
129
130         p->stride=10000/p->tickets;
131         p->pass=p->stride;
132
133         p->ticks=0;
```

- Scheduler() function is modified to include lottery and stride scheduling based on the preprocessor directives.

- **Lottery scheduling:**

- If the preprocessor directive is 'LOTTERY', then this section will be executed by the scheduler function.
- Lottery scheduling is based on the random number (lottery) drawn. The process holding the drawn number will be selected for scheduling.
- First, we need to get the total number of tickets held by all the processes. This is needed so that the number drawn does not exceed this total. This is obtained by adding the tickets held by each process that is in a 'RUNNABLE' state.
- Second, we used the rand() function to get the random number, which is used to draw a random winning number. If we divide any number by the maximum number of tickets, then the remainder will be always less than the maximum number of tickets. Hence, dividing the random number generated by the rand() function by maximum number of tickets will always give a remainder, which is less than the maximum number of tickets. This remainder is the winning number used to select the process to be scheduled. Since we are dividing by the maximum number of tickets, we need to ensure that the maximum number of tickets is not equal to zero to avoid a 'divide by zero' error.

- To select the process holding the winning number, a running\_total, which is a cumulative total, is obtained by adding the number of tickets held by the runnable processes up until the running point. If this running total is greater than the random winning number, then that process is the winner and will be scheduled. If not, the next process will be selected to check for a winning ticket, until the end of all the available runnable processes.
- The 'ticks' count is incremented whenever a process is selected for scheduling.

```
#if defined(LOTTERY)
int win_num;
int total_num_tickets = 0;
// Get the total number of tickets by adding tickets from each process
for(p = proc; p < &proc[NPROC]; p++) {
    acquire(&p->lock);
    if(p->state == RUNNABLE) {
        total_num_tickets += p->tickets;
        release(&p->lock);
    }
}
// Ensure that total_num_tickets not equal to zero as we are dividing by it to get the remainder
if (total_num_tickets != 0) {
    //select the winner by calling rand() function. Reminder (mod) is the winning number.
    win_num = rand() % total_num_tickets;
    //navigate the processes by counting the tickets until a process holding the ticket greater than winning number
    int running_total = 0;
    for(p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);
        if(p->state == RUNNABLE) {
            running_total += p->tickets;
            //if sum of tickets (running total) is > than winning number, winner is found.
            if (running_total > win_num) {
                p->state = RUNNING;
                c->proc = p;
                p->ticks++;
                switch(&c->context, &p->context);
                // Process is done running for now.
                // It should have changed its p->state before coming back.
                c->proc = 0;
                release(&p->lock);
                break;
            }
        }
        release(&p->lock);
    }
}
```

#### ○ Stride scheduling:

- If the preprocessor directive is 'STRIDE', then this section will be executed by the scheduler function.
- First, get the minimum pass value by iterating through each 'RUNNABLE' process.

- Then, find the 'RUNNABLE' process with that minimum pass value and select it for scheduling. Once selected, increment its pass value by the stride value and increment its 'ticks' count.

```
#elif defined(STRIDE)
    // Stride scheduling
    // select the process with the lowest pass value
    int min_pass = 2147483646;
    for(p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);
        if(p->state == RUNNABLE && p->pass < min_pass) {
            if(p->pass < min_pass){
                min_pass = p->pass;
            }
        }
        release(&p->lock);
    }
    // printf("min_pass: %d\n", min_pass);
    for(p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);
        if(p->state == RUNNABLE && p->pass == min_pass) {
            // Switch to chosen process. It is the process's job
            // to release its lock and then reacquire it
            // before jumping back to us.
            p->state = RUNNING;
            c->proc = p;
            p->pass += p->stride;
            p->ticks++;
            swtch(&c->context, &p->context);

            // Process is done running for now.
            // It should have changed its p->state before coming back.
            c->proc = 0;
        }
        release(&p->lock);
    }
}
```

- **Round Robin:**

- If the preprocessor directive is neither 'LOTTERY' nor 'STRIDE', the default scheduler (Round Robin) will run.
- Increment the 'ticks' count each time a process is scheduled to run.

540

p->ticks++;

- The **print\_statistics()** function was created, which will print the statistics of the schedules (PID, name, tickets, and ticks) in the format requested.

```
void print_statistics(void){
    struct proc *p;
    #if defined(LOTTERY)
        printf("Lottery Scheduling \n");
    #elif defined(STRIDE)
        printf("Stride Scheduling \n");
    #else
        print("Default: Round Robin \n");
    #endif
    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        //if(p->state != UNUSED || p->state != USED){
        if(p->state != UNUSED && p->state != USED){
            printf("%d(%s): tickets: %d, ticks: %d\n",p->pid,p->name,p->tickets,p->ticks);
        }
        release(&p->lock);
    }
}
```

- The **rand()** function (<https://stackoverflow.com/a/7603688>) was created as given in Lab2 instructions, which returns an unsigned random number in the short range when called.

- **kernel/defs.h:**

- The print\_statistics() and rand() functions were declared in the proc.c section.

```
112      void                print_statistics(void);
113      unsigned short rand();//CS202 Lab2
```

- **user/usys.pl:**

- The user space syscall interface is updated by creating entries for syscalls 'sched\_statistics' and 'sched\_tickets' in usys.pl

```
42      entry("sched_statistics");
43      entry("sched_tickets")
```

---



- **user/user.h:**

- User system calls (sched\_statistics and sched\_tickets) are updated in the user-space system call interface user.h.

```
48      int sched_statistics(void);  
49      int sched_tickets(int);
```

- **user/lab2.c:**

- This file provided in the lab2 instruction contains the code to test the sched\_statistics, sched\_tickets system calls and test the LOTTERY, STRIDE and Round Robin scheduling.

- **Makefile:**

- Makefile is updated by appending "\$U/\_lab2\" to UPROGS.

```
139      $U/_lab2\
```

- This file is also updated to include the preprocessor directives.

```
73      LAB2 = RR  
74      CFLAGS += -D$(LAB2)
```

---

### **Testing:**

The system calls (sched\_statistics and sched\_tickets) and schedules (LOTTERY, STRIDE, and Round Robin) were tested using lab2.c. To test LOTTERY, the code was compiled with a preprocessor directive using 'make clean;make lab2=LOTTERY'. To test STRIDE, the code was compiled with a preprocessor directive using 'make clean;make lab2=STRIDE'. To test Round Robin (default), the code was compiled with a preprocessor directive using 'make clean;make'.

### **Lottery scheduling output:**

xv6 kernel is booting

init: starting sh

\$ lab2 100 3 30 20 10

Lottery Scheduling

1(init): tickets: 10000, ticks: 25

2(sh): tickets: 10000, ticks: 13

3(lab2): tickets: 10000, ticks: 105

4(lab2): tickets: 30, ticks: 50

5(lab2): tickets: 20, ticks: 31

6(lab2): tickets: 10, ticks: 19

Stride scheduling output:

xv6 kernel is booting

init: starting sh

\$ lab2 100 3 30 20 10

Stride Scheduling

1(init): tickets: 10000, ticks: 24

2(sh): tickets: 10000, ticks: 13

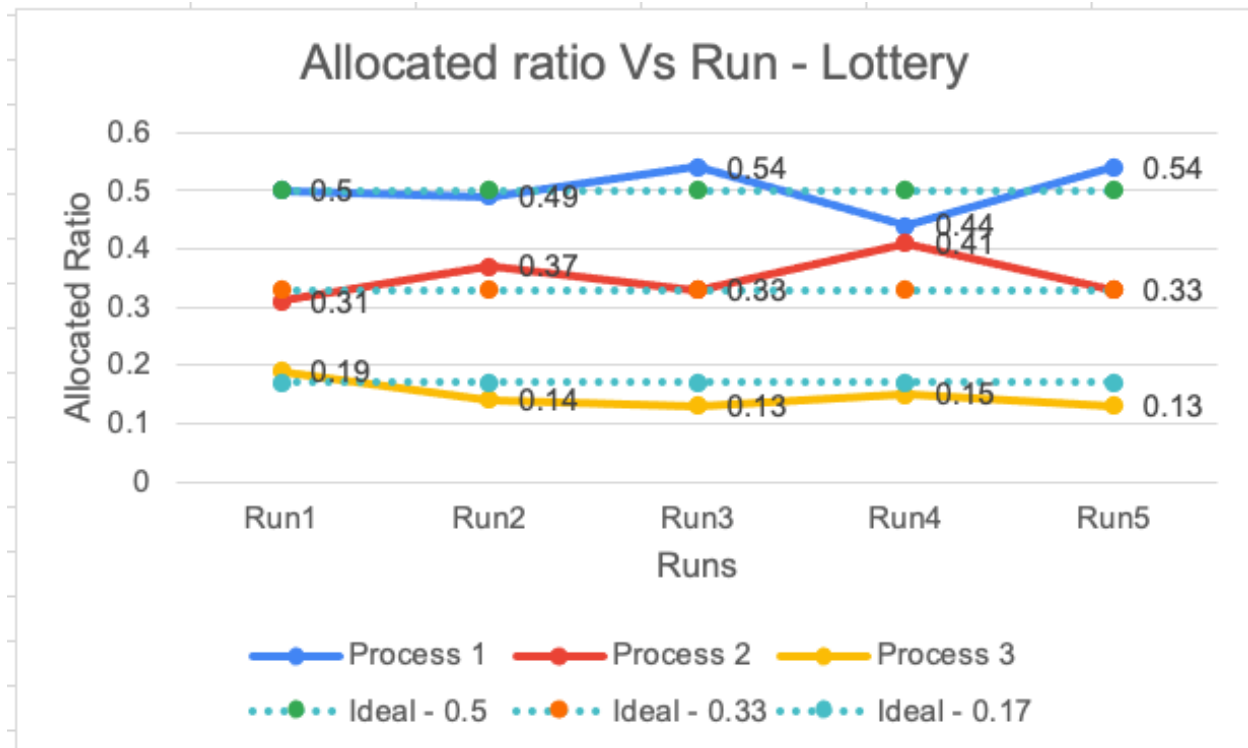
3(lab2): tickets: 10000, ticks: 88

4(lab2): tickets: 30, ticks: 50

5(lab2): tickets: 20, ticks: 33

6(lab2): tickets: 10, ticks: 17

|                     |       |                 |       |                 |       |                 |       |                 |       |                 |       |  |
|---------------------|-------|-----------------|-------|-----------------|-------|-----------------|-------|-----------------|-------|-----------------|-------|--|
| Number of processes | 3     |                 |       |                 |       |                 |       |                 |       |                 |       |  |
| Total ticks         | 100   |                 |       |                 |       |                 |       |                 |       |                 |       |  |
| Lottery (30/20/10)  | Run 1 | Allocated ratio | Run 2 | Allocated ratio | Run 3 | Allocated ratio | Run 4 | Allocated ratio | Run 5 | Allocated ratio | Ideal |  |
| Process 1           | 50    | 0.5             | 49    | 0.49            | 54    | 0.54            | 44    | 0.44            | 54    | 0.54            | 0.5   |  |
| Process 2           | 31    | 0.31            | 37    | 0.37            | 33    | 0.33            | 41    | 0.41            | 33    | 0.33            | 0.33  |  |
| Process 3           | 19    | 0.19            | 14    | 0.14            | 13    | 0.13            | 15    | 0.15            | 13    | 0.13            | 0.17  |  |
| Total actual ticks  | 100   | 1               | 100   |                 | 100   |                 | 100   |                 | 100   |                 |       |  |
| Stride (30/20/10)   | Run 1 | Allocated ratio | Run 2 | Allocated ratio | Run 3 | Allocated ratio | Run 4 | Allocated ratio | Run 5 | Allocated ratio | Ideal |  |
| Process 1           | 50    | 0.5             | 50    | 0.5             | 50    | 0.5             | 50    | 0.5             | 50    | 0.5             | 0.5   |  |
| Process 2           | 33    | 0.33            | 33    | 0.33            | 33    | 0.33            | 33    | 0.33            | 33    | 0.33            | 0.33  |  |
| Process 3           | 17    | 0.17            | 17    | 0.17            | 17    | 0.17            | 17    | 0.17            | 17    | 0.17            | 0.17  |  |
| Total actual ticks  | 100   |                 | 100   |                 | 100   |                 | 100   |                 | 100   |                 |       |  |



As we can see, for LOTTERY scheduling, the allocated ratio for each process varies for each run. For the 5 runs, for process 1, the allocated ratio varies from 0.44 to 0.54. For process 2, it varies from 0.31 to 0.41, and for process 3, it varies from 0.13 to 0.19. Ideally, the allocated ratio should be 0.5, 0.33, and 0.17 for process 1, process 2, and process 3 in each run. This implies that LOTTERY scheduling is non-deterministic and only probabilistically fair. This is due to the random function used to select the winner.

On the other hand, for STRIDE scheduling, the allocated ratio for each process remains the same for each run. For the 5 runs, the allocated ratio for process 1, process 2, and process 3 was 0.5, 0.33, and 0.17, respectively, which is equal to the expected ideal allocation. This implies that STRIDE scheduling is deterministic, and fairness is achieved completely. This is due to the stride and the pass values. The stride value will be smaller for higher priority tasks with a larger number of tickets than the ones with lower priority. The pass value, which gets incremented by the stride value for each tick, is like a virtual clock, and it increases slowly for the process with

higher priority (tickets) and faster for lower priority (tickets). It helps in achieving the fairness and getting the deterministic allocations.

### **Part3 - Experiment:**

We conducted the experiment for the lottery and stride scheduling as instructed. For 4 processes with 8/4/2/1 allocation, we ran for 5 different quanta (20,40,60,80, and 100). For each quantum, we ran for a minimum of 5 times to get the average. The screenshot below shows the same for lottery and stride scheduling.

#### **8/4/2/1 data for different runs to get average – lottery scheduling:**

| <b>Ticks</b>           |    | <b>20</b>  | <b>Run1</b> | <b>Run2</b> | <b>Run3</b> | <b>Run4</b> | <b>Run5</b> | <b>Run6</b> | <b>Average</b> |
|------------------------|----|------------|-------------|-------------|-------------|-------------|-------------|-------------|----------------|
| <b>Lottery 8/4/2/1</b> | P1 | 14         | 7           | 8           | 10          | 10          | 12          |             | 10.17          |
|                        | P2 | 3          | 7           | 6           | 5           | 6           | 4           |             | 5.17           |
|                        | P3 | 2          | 4           | 3           | 3           | 2           | 3           |             | 2.83           |
|                        | P4 | 1          | 2           | 3           | 2           | 2           | 1           |             | 1.83           |
| <b>Ticks</b>           |    | <b>40</b>  |             |             |             |             |             |             | <b>40.00</b>   |
|                        | P1 | 15         | 17          | 25          | 21          | 22          | 23          |             | 20.50          |
|                        | P2 | 15         | 16          | 6           | 9           | 11          | 8           |             | 10.83          |
|                        | P3 | 7          | 6           | 5           | 5           | 3           | 8           |             | 5.67           |
|                        | P4 | 3          | 2           | 4           | 5           | 4           | 1           |             | 3.17           |
| <b>Ticks</b>           |    | <b>60</b>  |             |             |             |             |             |             | <b>60.00</b>   |
|                        | P1 | 39         | 28          | 36          | 29          | 37          | 26          |             | 32.50          |
|                        | P2 | 11         | 18          | 12          | 17          | 8           | 15          |             | 13.50          |
|                        | P3 | 8          | 10          | 9           | 7           | 12          | 14          |             | 10.00          |
|                        | P4 | 2          | 4           | 3           | 7           | 3           | 5           |             | 4.00           |
| <b>Ticks</b>           |    | <b>80</b>  |             |             |             |             |             |             | <b>80.00</b>   |
|                        | P1 | 47         | 41          | 43          | 41          | 51          |             |             | 44.60          |
|                        | P2 | 18         | 18          | 19          | 20          | 14          |             |             | 17.80          |
|                        | P3 | 9          | 13          | 12          | 14          | 12          |             |             | 12.00          |
|                        | P4 | 6          | 8           | 6           | 5           | 3           |             |             | 5.60           |
| <b>Ticks</b>           |    | <b>100</b> |             |             |             |             |             |             | <b>100.00</b>  |
|                        | P1 | 53         | 54          | 58          | 54          | 55          |             |             | 54.80          |
|                        | P2 | 12         | 18          | 17          | 31          | 18          |             |             | 19.20          |
|                        | P3 | 10         | 13          | 18          | 12          | 21          |             |             | 14.80          |
|                        | P4 | 5          | 5           | 7           | 3           | 6           |             |             | 5.20           |

### 8/4/2/1 data for different runs to get average – Stride scheduling:

| Ticks          | 20  | Run1 | Run2 | Run3 | Run4 | Run5 | Run6 | Average |
|----------------|-----|------|------|------|------|------|------|---------|
| Stried 8/4/2/1 | P1  | 10   | 10   | 10   | 10   | 10   |      | 10      |
|                | P2  | 5    | 5    | 5    | 5    | 5    |      | 5       |
|                | P3  | 3    | 3    | 3    | 3    | 3    |      | 3       |
|                | P4  | 2    | 2    | 2    | 2    | 2    |      | 2       |
| Ticks          | 40  |      |      |      |      |      |      |         |
|                | P1  | 21   | 21   | 21   | 21   | 21   |      | 21      |
|                | P2  | 11   | 11   | 11   | 11   | 11   |      | 11      |
|                | P3  | 6    | 6    | 6    | 6    | 6    |      | 6       |
|                | P4  | 3    | 3    | 3    | 3    | 3    |      | 3       |
| Ticks          | 60  |      |      |      |      |      |      |         |
|                | P1  | 32   | 32   | 32   | 32   | 32   |      | 32      |
|                | P2  | 16   | 16   | 16   | 16   | 16   |      | 16      |
|                | P3  | 8    | 8    | 8    | 8    | 8    |      | 8       |
|                | P4  | 4    | 4    | 4    | 4    | 4    |      | 4       |
| Ticks          | 80  |      |      |      |      |      |      |         |
|                | P1  | 42   | 42   | 42   | 42   | 42   |      | 42      |
|                | P2  | 21   | 21   | 21   | 21   | 21   |      | 21      |
|                | P3  | 11   | 11   | 11   | 11   | 11   |      | 11      |
|                | P4  | 6    | 6    | 6    | 6    | 6    |      | 6       |
| Ticks          | 100 |      |      |      |      |      |      |         |
|                | P1  | 53   | 53   | 53   | 53   | 53   |      | 53      |
|                | P2  | 27   | 27   | 27   | 27   | 27   |      | 27      |
|                | P3  | 14   | 14   | 14   | 14   | 14   |      | 14      |
|                | P4  | 7    | 7    | 7    | 7    | 7    |      | 7       |

Similarly, for 4 processes with 1/1/1/1 allocation, we ran for 5 different quanta (20,40,60,80, and 100). For each quantum, we ran for a minimum of 5 times to get the average. The screenshot below shows the same for lottery and stride scheduling.

### 1/1/1/1 data for different runs to get average – lottery scheduling:

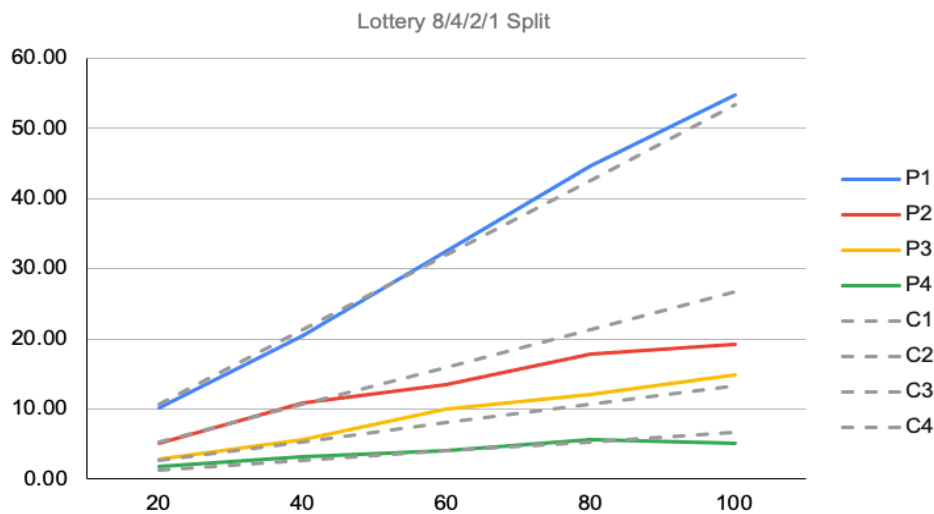
| Ticks           | 20  | Run1 | Run2 | Run3 | Run4 | Run5 | Run6 | Average |
|-----------------|-----|------|------|------|------|------|------|---------|
| Lottery 1/1/1/1 | P1  | 6    | 3    | 6    | 2    | 5    | 5    | 4.50    |
|                 | P2  | 5    | 9    | 4    | 7    | 4    | 6    | 5.83    |
|                 | P3  | 6    | 2    | 5    | 7    | 9    | 6    | 5.83    |
|                 | P4  | 3    | 6    | 5    | 4    | 2    | 3    | 3.83    |
| Ticks           | 40  |      |      |      |      |      |      | 40.00   |
|                 | P1  | 7    | 7    | 8    | 10   | 8    |      | 8.00    |
|                 | P2  | 11   | 10   | 7    | 13   | 10   |      | 10.20   |
|                 | P3  | 8    | 12   | 13   | 9    | 10   |      | 10.40   |
|                 | P4  | 14   | 11   | 12   | 8    | 12   |      | 11.40   |
| Ticks           | 60  |      |      |      |      |      |      | 60.00   |
|                 | P1  | 12   | 12   | 11   | 15   | 11   | 14   | 12.50   |
|                 | P2  | 21   | 13   | 16   | 17   | 20   | 16   | 17.17   |
|                 | P3  | 16   | 16   | 21   | 13   | 16   | 17   | 16.50   |
|                 | P4  | 11   | 19   | 12   | 15   | 13   | 13   | 13.83   |
| Ticks           | 80  |      |      |      |      |      |      | 80.00   |
|                 | P1  | 21   | 23   | 24   | 12   | 19   |      | 19.80   |
|                 | P2  | 20   | 22   | 15   | 25   | 22   |      | 20.80   |
|                 | P3  | 19   | 20   | 21   | 25   | 17   |      | 20.40   |
|                 | P4  | 20   | 15   | 20   | 18   | 22   |      | 19.00   |
| Ticks           | 100 |      |      |      |      |      |      | 100.00  |
|                 | P1  | 24   | 21   | 29   | 21   | 26   |      | 24.20   |
|                 | P2  | 27   | 24   | 29   | 30   | 31   |      | 28.20   |
|                 | P3  | 28   | 27   | 22   | 23   | 16   |      | 23.20   |
|                 | P4  | 21   | 28   | 20   | 26   | 25   |      | 24.00   |

**1/1/1/1 data for different runs to get average – stride scheduling:**

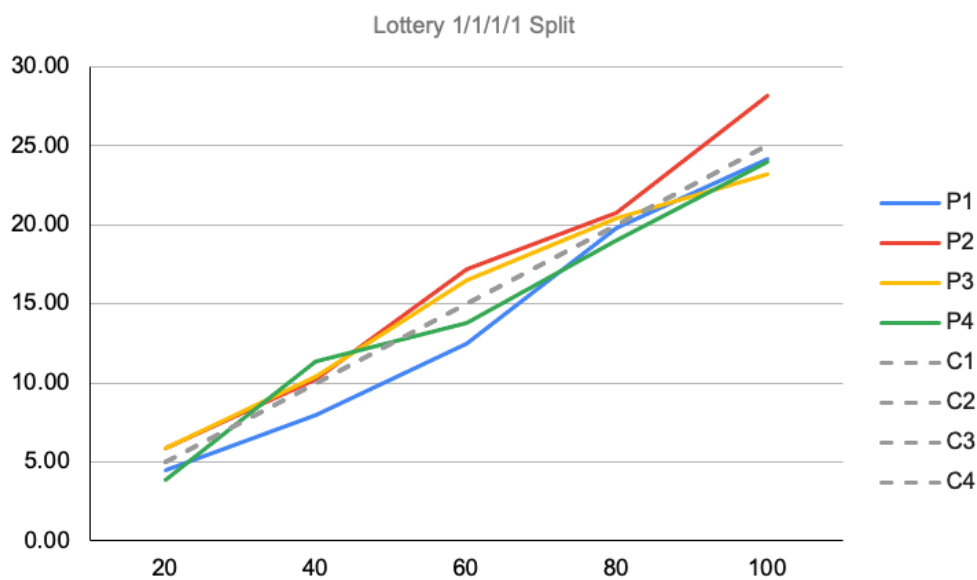
| Ticks          | 20  | Run1 | Run2 | Run3 | Run4 | Run5 | Run6 | Average |
|----------------|-----|------|------|------|------|------|------|---------|
| Stride 1/1/1/1 | P1  | 5    | 5    | 5    | 5    | 5    |      | 5       |
|                | P2  | 5    | 5    | 5    | 5    | 5    |      | 5       |
|                | P3  | 5    | 5    | 5    | 5    | 5    |      | 5       |
|                | P4  | 5    | 5    | 5    | 5    | 5    |      | 5       |
| Ticks          | 40  |      |      |      |      |      |      |         |
|                | P1  | 10   | 10   | 10   | 10   | 10   |      | 10      |
|                | P2  | 10   | 10   | 10   | 10   | 10   |      | 10      |
|                | P3  | 10   | 10   | 10   | 10   | 10   |      | 10      |
|                | P4  | 10   | 10   | 10   | 10   | 10   |      | 10      |
| Ticks          | 60  |      |      |      |      |      |      |         |
|                | P1  | 15   | 15   | 15   | 15   | 15   |      | 15      |
|                | P2  | 15   | 15   | 15   | 15   | 15   |      | 15      |
|                | P3  | 15   | 15   | 15   | 15   | 15   |      | 15      |
|                | P4  | 15   | 15   | 15   | 15   | 15   |      | 15      |
| Ticks          | 80  |      |      |      |      |      |      |         |
|                | P1  | 20   | 20   | 20   | 20   | 20   |      | 20      |
|                | P2  | 20   | 20   | 20   | 20   | 20   |      | 20      |
|                | P3  | 20   | 20   | 20   | 20   | 20   |      | 20      |
|                | P4  | 20   | 20   | 20   | 20   | 20   |      | 20      |
| Ticks          | 100 |      |      |      |      |      |      |         |
|                | P1  | 25   | 25   | 25   | 25   | 25   |      | 25      |
|                | P2  | 25   | 25   | 25   | 25   | 25   |      | 25      |
|                | P3  | 25   | 25   | 25   | 25   | 25   |      | 25      |
|                | P4  | 25   | 25   | 25   | 25   | 25   |      | 25      |

We plotted time (quanta) vs. cumulative quanta for each process and created 4 different charts, which are shown below (C1, C2, C3, and C4 are ideal values with dotted lines, and P1, P2, P3, and P4 are 4 processes).

### 8/4/2/1 Lottery:



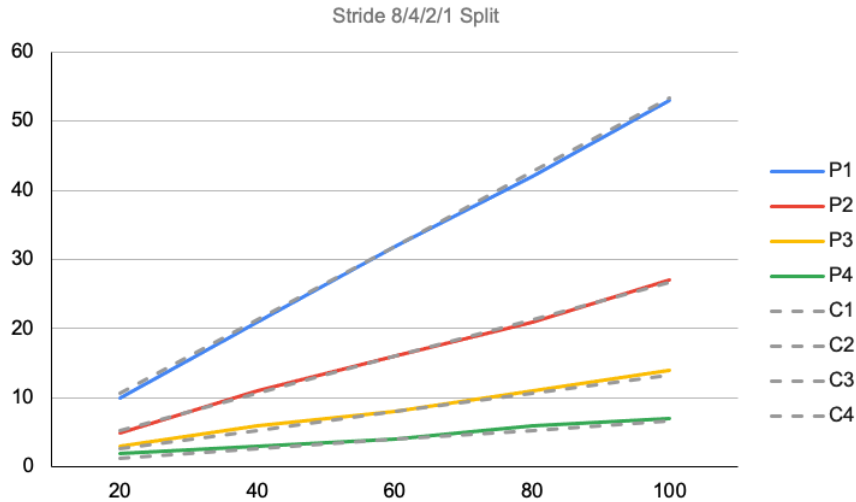
### 1/1/1/1 Lottery:



As we can see in the charts for 8/4/2/1 and 1/1/1/1 allocations above, lottery scheduling is not having the expected allocation for each process. Process 1 is not having much of a problem with 8/4/2/1 allocation. The 8/4/2/1 chart shows that at the lower quanta (like 20 and 40), each process has almost the ideal allocation, as there is not much deviation. However, for process 2, it is getting a lower allocation than the ideal value after 40 quanta. Process 4 is also having near-ideal values allocated. For 1/1/1/1 allocation, we can see the deviation from ideal values for each process, and it is random. As explained above, this implies that LOTTERY scheduling is non-

deterministic, and fairness is not achieved completely. It is only probabilistically fair. This is due to the random function used to select the winner.

### 8/4/2/1 Stride:



### 1/1/1/1 Stride:



On the other hand, STRIDE scheduling is deterministic and fair, which can be seen from the charts shown. For both 8/4/2/1 and 1/1/1/1 allocations, the cumulative quanta is matching with the ideal values for all the processes. As explained above, the stride scheduling is more deterministic and fairer. This is due to the stride and the pass values. The stride value will be smaller for higher-priority tasks with a larger number of tickets than the ones with lower priority. The pass value, which gets incremented by the stride value for each tick, is like a virtual clock, and



it increases slowly for the process with higher priority (tickets) and faster for lower priority (tickets). It helps in achieving the fairness and getting the deterministic allocations.

**Contributions:**

The team discussed the high-level approach and decided to do the following by everyone.

1. Hunter Adomitis: sched\_statistics and sched\_tickets. Review both system calls (sched\_statistics and sched\_tickets), lottery, stride, and RR scheduling.
2. Iftikhar Kalanther: LOTTERY scheduling. Review both system calls (sched\_statistics and sched\_tickets), lottery, stride, and RR scheduling.
3. Joseph D'Angelo: STRIDE scheduling Review both system calls (sched\_statistics and sched\_tickets), lottery, stride, and RR scheduling.

All 3 members ran Part-3 experiment. The team collaborated to create the final report, which was reviewed by each team member before submission.